Response to the Reviewers – draft of Giuseppe's points

Reviewer #1

3. Provide valid numerical bounds for M and Q_{max}, and clarify the saturation convention when cumulative load equals vehicle capacity in constraints (17)–(18).

We thank the Reviewer for this comment. Let $\bar{Q} = \max_{s\in S} \sum_{j=1}^{n} \left(d_j + \sum_{t\in T} \delta_{jt}^s\right)$ be the total waste that can appear in a scenario, so that $0 \le Q_j^s \le \bar{Q}$ for every node and scenario. Then:

- constraints (10)–(13) are valid for $M \ge \bar{Q}$;
- constraints (17)–(18) are valid for $Q_{max} \ge \max\{C, \bar{Q} - C\}$: (17) forces $\rho_j^s = 1$ when $Q_j^s > C$ and (18) forces $\rho_j^s = 0$ when $Q_j^s < C$;
- in (7) the code uses T_{max}, which is valid because travel times are Euclidean and service times are zero.

In our instances $\bar{Q} \le 1024$, so the value $M = Q_{max} = 10^4$ used in the experiments is always valid. It is also not large enough to cause numerical problems: since the data are integers or multiples of 0.5, whenever $Q_j^s \ne C$ the term $|Q_j^s - C|/Q_{max}$ is at least $5 \cdot 10^{-5}$, above the integrality tolerance of Gurobi (10^{-5}). As a check, we re-solved a sample of 332 of the 548 instances solved to optimality (all those with $n \in \{5, 6\}$ and 35 random ones with $n \in \{7, 8\}$; 629 SM and EM runs in total) with the smallest valid values ($M = \bar{Q}$, $Q_{max} = \max\{C, \bar{Q} - C\}$): all optimal values coincide (relative difference below $5 \cdot 10^{-6}$), and computing times decrease slightly (-11% in total).

When $Q_j^s = C$, the vehicle is considered saturated, consistently with Section 3.2: it loads the whole demand of j and returns to the depot. In this case (17)–(18) leave ρ_j^s free, but setting $\rho_j^s = 1$ is never more costly (triangle inequality), so the optimal value is not affected. In our 597 SM solutions, whenever $Q_j^s = C$ and j is followed by another collection point, the solver sets $\rho_j^s = 1$; $\rho_j^s = 0$ only occurs when j is the last point of its route, where the two choices have the same cost.

Changes in the manuscript. In constraint (7), M is replaced by T_{max}, which is the value used in our code and the same constant already used in (8); M remains the large constant of (10)–(13). In Table 3 the notation is now defined accordingly. After constraints (17)–(18) we add:

> Constraints (??)-(??) and (??)-(??) are valid for $M \ge \bar{Q}$ and $Q_{max} \ge \max\{C, \bar{Q} - C\}$, respectively, where $\bar{Q} = \max_{s\in S} \sum_{j=1}^{n} (d_j + \sum_{t\in T} \delta_{jt}^s)$ bounds any cumulative load. In our experiments we set $M = Q_{max} = 10^4$; re-solving a sample of 332 instances with the smallest valid values gives the same optimal values. When $Q_j^s = C$ the vehicle is considered saturated; (??)-(??) let in this case ρ_j^s, but $\rho_j^s = 1$ does not imply higher cost, thus the optimal value is not affected.

Internal note (to be removed): Comment 2 (Francesca) discusses why (19) does not force $y = 1$; the triangle-inequality argument above is the same one, so the two answers should cross-reference each other. Script used for the empirical check: `wm-collection-stochastic` repository, files `results/sm/*`.

4. Reconcile the reported average VSS of 21.33% with Table 5. Weighting the displayed group averages by their respective numbers of solved instances gives approximately 27.02%. Please verify the calculation and update the discussion and conclusions accordingly.

We thank the Reviewer for checking the figures so carefully. The Reviewer's computation is correct: weighting the group averages of Table 5 (Table 6 in the revised manuscript) by the number of solved instances gives 27.02%. We traced the inconsistency back to two errors in the aggregation of the results, both now corrected:

1. the group averages VSS_a (and gap_a) in Table 6 were computed only over the instances with a strictly positive value, i.e. instances with $VSS = 0$ (where EM and SM give the same solution value) were excluded from the average. This inflates the group averages, and therefore also the 27.02% obtained by weighting them;

2. the overall value 21.33% was computed over the 548 instances solved with an optimality gap of at most 0.1% (the same set used in Section 5.3), and not over the 597 instances as stated in the text.

In the revised manuscript Table 6 is recomputed over *all* instances with a feasible solution, including those with $VSS = 0$. We also add a column with the number of instances with a positive VSS. The overall figures are now:

- average VSS over the 597 instances with a feasible solution: **19.65%**. This value coincides with the average of the revised group values weighted by the number of solved instances;

- average VSS over the 548 instances solved with gap $\leq 0.1\%$, for which both SM_{obj} and EM_{obj} are certified: **21.33%**. For the remaining 49 instances (all with $n = 8$) the average is 0.86%;

- the VSS is positive in 356 of the 597 instances (59.6%), with an average of 32.96% over these instances, and is zero in 241 instances. The distribution is strongly skewed (median 0.12%, maximum 581.79%): large values are concentrated in the groups with large capacity and few collection points.

The qualitative conclusions do not change. The revised averages still increase with the vehicle capacity for every value of n, and uncertainty is relevant for a large share of the instances. We have updated the discussion of Section 5.2 and the conclusions accordingly. For full transparency, the per-instance values and the script that recomputes Table 6 are available in the public repository (see our answer to Reviewer 3).

Changes in the manuscript. Table 6 is replaced by:

						SM_{obj}				EM_{obj}			
n	C	feasible	VSS_a	VSS_M	$\#VSS{>}0$	$time_a$	$time_M$	gap_a	gap_M	$time_a$	$time_M$	gap_a	gap_M
5	150	50/60	1.69	20.04	16	1.66	3.58	0.00	0.00	0.49	1.12	0.00	0.00
5	200	50/60	50.54	581.79	24	1.01	2.56	0.00	0.00	0.28	0.70	0.00	0.00
5	250	50/60	74.28	538.26	30	0.53	1.00	0.00	0.00	0.18	0.40	0.00	0.00
6	150	49/60	0.35	7.36	19	16.73	63.55	0.00	0.00	3.71	10.11	0.00	0.00
6	200	49/60	3.29	23.34	27	6.36	22.71	0.00	0.00	1.09	3.40	0.00	0.00
6	250	49/60	61.11	198.09	41	0.91	2.97	0.00	0.00	0.30	0.82	0.00	0.00
7	150	50/60	0.59	3.01	32	317.16	1022.41	0.00	0.00	63.04	144.67	0.00	0.00
7	200	50/60	3.88	24.77	39	197.73	520.44	0.00	0.00	26.57	85.40	0.00	0.00
7	250	50/60	28.20	351.36	31	55.41	236.39	0.00	0.00	3.41	13.08	0.00	0.00
8	150	50/60	0.15	2.35	30	2507.71	3600.39	0.47	0.98	1631.95	3600.13	0.14	0.76
8	200	50/60	2.22	12.99	40	1777.46	3600.26	0.23	0.97	455.52	1956.51	0.00	0.00
8	250	50/60	9.66	131.13	27	652.68	3600.15	0.07	0.91	106.84	1175.79	0.00	0.00
all		597/720	19.65	581.79	356								

Table 6: Results for groups of instances with the same number of collection points and vehicle capacity. Averages are computed over all instances with a feasible solution (including those with $VSS = 0$). Gaps are relative MIP gaps (fractions).

In Section 5.2 the column "solved" becomes "feasible", the description of VSS_a becomes "the average percentage value for VSS over the instances with a feasible solution, including those for which $VSS = 0$ (VSS_a)"; we add "the number of instances with a strictly positive VSS ($\#VSS{>}0$)"; and the last sentence of the section is replaced by:

> Finally, over the 597 instances for which a feasible solution was found, the average percentage VSS is 19.65%. It rises to 21.33% on the 548 instances solved with an optimality gap not greater than 0.1%, for which the VSS is computed from certified values. The VSS is strictly positive in 356 instances (59.6%), where it averages 32.96%, while in the remaining 241 instances the expected-value solution performs as well as the stochastic one. The distribution is therefore highly skewed (median 0.12%), with the largest values concentrated in the instances with few collection points and large capacity.

In the Conclusions, the sentence reporting the average VSS now reads: "On this subset, the average estimated VSS-type measure was 19.65% (21.33% on the 548 instances solved to near-optimality), and it was positive in about 60% of the instances, suggesting that explicitly accounting for demand uncertainty can provide benefits over the expected-value approach."

Internal note (to be removed): (a) The "Finally, we report that on all 597 solved instances . . . 21.33" sentence is also Reviewer 3's last comment on the Conclusions (Simona): the answer there should match the numbers above. (b) The old gap_a column also averaged only positive gaps: e.g. $n = 8$, $C = 150$ goes from 0.68 to 0.47. The gaps are fractions (0.47 = 47%); the caption now says so. (c) All 123 "not solved" instances are in fact *proven infeasible* by Gurobi (status INFEASIBLE in the logs), not instances without a primal bound within the time limit as stated in Section 5.2. 120 of them have the depot in the lower-left corner and $T_{max} = 15$, so the round trip to the farthest collection point is longer than T_{max}. The other 3 are `I3_N6_M3_T15_C{150,200,250}_DepCentral_s1`. This directly answers Reviewer 3 ("distinguish instances for which no feasible solution was found from those proven infeasible", Simona), and the definition of "solved" in Section 5.2 must be corrected. I used "feasible" in the revised table header for this reason.

Reviewer #2

1. [. . .] What is the mathematical difference and therefore the challenge associated with intra-day with respect to those that assume stochasticity across days? This needs to be clarified from a mathematical formulation point of view [. . .] May be better to add another column to distinguish this aspect across papers?

We thank the Reviewer for this important comment, which helped us sharpen the positioning of the paper. In the revised manuscript we explain the difference from the point of view of the formulation (new paragraph at the end of Section 3.3), and Table 1 has two new columns. The key difference is that, in the intraday setting, the quantity to be collected at a point is a *decision-dependent* random variable. It depends on the time at which the vehicle reaches the point, and hence on the sequence of the route.

- *Interday models* (e.g. Markov et al., 2020) use the day as the elementary period. If collection point j is served on day τ, the quantity collected is a random variable $\xi_{j\tau}$ whose distribution depends on the day, but not on the position of j in the route or on the time of the visit within that day. For a given day, the load of a vehicle is $\sum_{j \in R} \xi_{j\tau}$. It depends only on the *set* R of points assigned to the route, so the probability of a capacity failure is the same for every sequence of R, and the sequence only affects the travel cost.

- *In our intraday model*, the quantity collected at j is $q_j^s = d_j + \sum_t \sum_{\tau \le t} \delta_{j\tau}^s U_{jt}$: it depends on the period U_{jt} in which j is visited. Through (7)–(8) this period is determined by the arrival time $\bar{T}_j$, and therefore by the sequence of the route. Three consequences follow, none of which arises in interday models:

  1. the cumulative load Q_j^s in (10)–(11) couples routing, timing and the random data. The same set of points may or may not saturate the vehicle depending on the order in which the points are visited and on when the route starts, so the sequence affects both feasibility and the recourse, not only the cost;

2. the daily amount $d_j + \sum_t \delta^s_{jt}$ is split by the visit time into a collected part q^s_j and a leftover part L^s_j (16). Visiting later reduces the leftover penalty but increases the load and the risk of saturation. This trade-off is internal to a single route, and it is what makes the choice of the visit period a genuine decision;
3. since the recourse (skipping after saturation) is triggered by a load that depends on the arrival times, the expected recourse cost is a function of the whole schedule, not of the assignment alone.

- *Finer discretisation of interday models.* Splitting the day into shorter periods in a multi-period (inventory-)routing model does not reproduce this structure. In such models each route is performed within one period and accumulation is tracked between periods. A single route spanning several periods, in which the load accumulates while the vehicle is travelling, cannot be represented. Our formulation introduces exactly this link through constraints (7)–(8) and (10)–(11), which tie the time slot of each visit to the position of the point along the route.

Table 1 now includes two columns: "Demand evolution" (static / interday / intraday) and "Collected quantity depends on service time" (no / day of visit / time of visit within the route). For example, Markov et al. (2020) is classified as *interday / day of visit*, and our paper as *intraday / time of visit within the route*.

Text added at the end of Section 3.3:

> It is worth emphasising what distinguishes the intraday setting from models in which demand varies across days. In the latter, the quantity collected at a point on a given day does not depend on when, within the route, the point is visited. The load of a vehicle, and hence its probability of capacity failure, depends only on the set of points assigned to the route. In the WCRP-TVSD, instead, the quantity $q^s_j = d_j + \sum_{t \in T} \sum_{\tau=1}^{t} \delta^s_{j\tau} U_{jt}$ available at point j is a decision-dependent random variable. It depends on the time slot of the visit, which is linked by (7)–(8) to the arrival time and therefore to the sequence of the route. As a consequence, (i) the cumulative loads (10)–(11) couple routing, scheduling and uncertainty; (ii) the daily amount generated at each point is split by the visit time into a collected part and a leftover part (16), creating a trade-off between leftover penalty and saturation risk within each route; (iii) the expected recourse cost depends on the full schedule rather than on the assignment of points to vehicles. These features cannot be obtained by simply refining the time discretisation of a multi-period model, where each route is executed within a single period.

Internal note (to be removed): Francesca follows up on this point. Please verify the classification of the other papers in Table 1 against the original articles. My proposal, based only on how they are described in Section 2.2: Laporte et al. (2010), Gruler et al. (2017), Liao et al. (2025): static / no. Gruler et al. (2020), Nguyen et al. (2025), Xu et al. (2025): static demand (travel-time uncertainty) / no. Nolz et al. (2014), Markov et al. (2020), Spinelli et al. (2025): interday / day of visit. Rabbani et al. (2019): to be checked. Yu et al. (2021), requested by Reviewer 3, could be added as deterministic / time of visit.

Reviewer #3

6. [Computational results] Please make the generated instances and accompanying documentation available in a public repository. The documentation should describe the file format, parameters, units, demand-generation procedure and random seeds. Providing the instance-generation code would further support reproducibility.

We thank the Reviewer for this suggestion, which we have fully implemented. All the material is now available in the public repository `https://github.com/gstecca/wm-collection-stochastic`, which contains:

- the 720 instances and, for each of them, the expected-value instance used by the EM and by the policies;
- the instance-generation code. Running it regenerates all the instance files exactly; we verified the match sheet by sheet against the files used in the paper;
- the per-instance solutions of the SM, of the EM, of the EM routes evaluated on the scenarios (EM_{obj}), and of the 12 policies on the 178 instances of Section 5.3 (routes, visit periods, arrival times, loads, skipped arcs and the components of the objective);
- summary tables with one row per instance (and per instance–policy pair), the ML dataset (features and policy values), and a script that recomputes Table 6;
- the optimisation code (Python/Gurobi).

The documentation (README) describes the file format sheet by sheet, the parameters and their units, and the demand-generation procedure, including the order of the random draws, the distribution of each increment class, the assignment of collection-point types and the dependence structure. It also lists the random seeds: Python's `random` module is reset with seeds 0–4 before each instance. As explained there, for a given number of collection points and seed, the locations and demands are shared by the 36 combinations of fleet size, capacity, time horizon and depot position. The repository is referenced in Section 5.1 and in a new Data Availability statement:

> **Data availability.** The instances, the instance generator, the detailed results of all experiments and the code used in this paper are publicly available at `https://github.com/gstecca/wm-collection-stochastic`.

Internal note (to be removed): Section 5.1 of the manuscript does *not* describe the generator correctly, and this must be fixed together with Reviewer 3's bullet "document the instance-generation procedure more fully" (Simona). The paper says that the baseline is $U[10, 30]$ for both d_j and δ, multiplied by 1.5/0.5 according to the type. In the code: (i) $d_j \sim \text{round}(U[10, 30])$ for *all* types (the type multiplier is commented out, so "d_j low/average/high" in the patterns is not true); (ii) the increments are $\text{round}(U[0, 25])$ (low), $\text{round}(U[25, 50])$ (average) and $\text{round}(U[50, 75])$ (high), with expected values 12.5/37.5/62.5; (iii) types are drawn uniformly in $\{1, 2, 3\}$ (school, home, restaurant); (iv) increments are independent across points, slots and scenarios. A corrected paragraph is given below.

Proposed corrected text for Section 5.1 (to be agreed with Simona):

> Each collection point is assigned one of three types, chosen with equal probability, which determine the pattern of its demand increments over the three time slots: type 1 (e.g. school) high–low–low, type 2 (e.g. residential) average–high–low, type 3 (e.g. restaurant) low–low–average. Initial demands are drawn as $d_j = \text{round}(U[10, 30])$. In each scenario, the increment of a collection point in a time slot is drawn as $\text{round}(U[0, 25])$, $\text{round}(U[25, 50])$ or $\text{round}(U[50, 75])$ when the corresponding level is low, average or high, respectively, with expected values 12.5, 37.5 and 62.5 used in the EM. Increments are independent across collection points, time slots and scenarios, and the five scenarios are equiprobable. Travel costs and times are the Euclidean distances, service times are zero, the unit penalty is $p = 10$, and the three time slots have equal length $T_{max}/3$. Each instance is generated with Python's `random` module initialised with a seed in $\{0, \ldots, 4\}$.